

Frontend Build Plan

Sovereign On-Premise Agentic AI Workbench — PS 26117 Owner: Backend Dev B (per PRD v2 team split) · Stack locked: React + Vite + Tailwind

1. What this frontend actually has to prove to a judge

Not "does the app look nice" — it has to make five invisible things visible, live:

- * The agent is actually planning and acting, not just returning a chat reply — every step shows up, not just the final answer.
- * Model routing is real — which model handled this, and why.
- * Nothing left the device — the sovereignty monitor is on-screen the entire demo, not a slide shown once.
- * A human approves, the AI doesn't silently decide — the approval gate has to be a real, visible step, not implied.
- * Real files come out the other end — artifacts are downloadable, not just described in text.

If a component doesn't serve one of these five, it's not worth building for the demo. Keep that filter running the whole time.

2. Stack (locked, per PRD FR-10)

Choice	Why
React + Vite	Fast dev loop, no build-config fighting
Tailwind	Skip hand-rolled CSS; speed over visual originality
No Next.js / SSR	One user, one machine, no SEO — SSR buys nothing here and costs setup time
Native EventSource (SSE), no extra library	<code>/api/chat</code> and the timeline both stream via SSE per the API contract — the browser API is enough, don't add a dependency for this
React Context + useReducer for shared state	A handful of panels need shared state (current task, artifacts, sovereignty counters, timeline) — nothing here needs Redux-level machinery
No routing library	Single-page workbench, no navigable routes — a <code>currentView</code> state variable is enough

Don't add anything to this list without a reason tied to §1 — every extra dependency is something that can break on a laptop the night before finale.

3. Layout (from PRD, this is the frozen shape)

Sidebar	Main: Conversation	Context Panel
New Task		Files
Documents	User message	Sources
Knowledge	↓	Artifacts
Artifacts	[Routing receipt badge]	P&ID Graph
Models	↓	
Activity	Agent step (live)	
	↓	
	Agent step (live)	
	↓	
	[Approval gate, if any]	
Agent Execution Timeline (always visible during a task)		
Sovereignty Monitor strip (always visible, entire session)		

The sovereignty strip isn't in the original PRD wireframe as a persistent element — add it as a thin, always-on footer/header bar, separate from the per-task execution timeline. It needs to survive across every screen and every task, not just show up during Workflow 3.

4. Core components

Component	Responsibility	Talks to
ConversationView	Renders the message stream + live agent steps as they arrive over SSE	POST <code>/api/chat</code> , then subscribes to task progress
RoutingReceiptBadge	Small inline badge: "Handled by: coding · qwen2.5-coder:7b"	Comes attached to each streamed response
ExecutionTimeline	Footer checklist, one line per audit-log entry, live-updating	GET <code>/api/tasks/{id}/timeline</code> (poll or SSE)
SovereigntyMonitor	Persistent strip: Internet Access / External API Calls / External Connections / Local Requests / Status	GET <code>/api/monitoring/sovereignty</code> , polled every 1–2s
ArtifactsPanel	List + download of generated files, grouped by task	GET <code>/api/artifacts/{id}</code>
ApprovalGate	Shows the pending recommendation, three buttons: Approve / Modify / Reject	POST <code>/api/tasks/{id}/approve</code>
PIDGraphViewer	Overlay image + a simple node/edge list next to it (click a node → highlight on image)	POST <code>/api/pid/analyze</code> , renders returned overlay URL + graph JSON

Component	Responsibility	Talks to
KnowledgeUpload	Drag-and-drop or file-picker for SOPs/manuals into the RAG corpus	POST <code>/api/knowledge/upload</code>
ModelsPanel	Read-only view of the registry + which model is currently resident	GET <code>/api/models</code>
FileDropzone	Upload a scan/image/code file to start a task	POST <code>/api/files/upload</code>

RoutingReceiptBadge and SovereigntyMonitor are the two components doing the most persuasion work per line of code — build them early and don't let them be the last thing polished.

5. State shape (rough)

```
{
  "currentView": "conversation | documents | knowledge | artifacts | models | activity",
  "activeTask": {
    "task_id": "...", "status": "...", "steps": [], "routing": {"model": "...", "reason": "..."},
    "artifacts": [], "pidGraph": null, "requiresApproval": true
  },
  "sovereignty": { "internetBlocked": true, "externalCalls": 0, "externalDns": 0, "externalConnections": 0, "localConnections": 0 },
  "knowledgeCorpus": [],
  "modelRegistry": []
}
```

Keep `activeTask` as one object updated by the SSE stream, not scattered pieces of state — it maps directly onto the backend's task-state schema, which makes debugging "why didn't the UI update" much easier.

6. Build order (don't wait on the backend to start)

Do this first, regardless of backend progress: stand up a tiny mock API (a 30-line FastAPI stub or even static JSON fixtures) that returns fake `/api/chat` SSE events, a fake sovereignty payload, and a fake artifact list. The real API contract is already frozen (PRD §8) — build against that shape now, swap the mock for the real backend later. This is what stops the frontend from being blocked on the other two people.

1. App shell: sidebar + main + context panel + footer layout, static, no data yet.
2. Sovereignty strip, wired to the mock — this is cheap and it's the highest-value component, get it running first.
3. Conversation view + SSE wiring, against mock streamed events.
4. Routing receipt badge, rendered from whatever the mock/task object returns.
5. Execution timeline, rendered from mock audit-log entries.
6. Artifacts panel, with real download links once real files exist (this can stay mocked longest — it's just a list + link).
7. Approval gate, wired to the mock approve endpoint.

8. Knowledge upload view.
9. P&ID graph viewer — build this against a static sample overlay image + hardcoded graph JSON first; wire to the real endpoint only once the CV pipeline is producing real output (this will be the last backend piece ready, per the main PRD's own sequencing).
10. Models panel (lowest priority — nice-to-have, cut first if time runs out).

Swap every mock call for the real API, one endpoint at a time, testing after each swap — don't do this all at once the night before the demo. Loading/error states: a visible "loading model..." state during hot-swap (per PRD §4.1 — swap isn't instant, the UI must not look frozen), a clear error state if a task fails verification past the retry limit.

7. Visual direction (rough, not a design system)

- Dark, technical, "workbench" feel over a bright consumer-app look — this is a tool for engineers, not a marketing site. Dark background, monospace for logs/timeline/routing receipts, a clean sans for conversation text.
- Status color coding, used consistently everywhere: green = verified/passed/air-gapped, amber = pending/in-progress, red = failed/blocked-and-logged. The sovereignty monitor's "BLOCKED" status should read as a good thing (green), not an alarm — first-time viewers can misread a red "blocked" as something going wrong.
- No animation beyond simple fades/spinners — nothing that could look flaky mid-demo.

8. Explicit non-goals for the frontend

- No login/auth screen beyond a single local session — matches the backend's single-tenant scope.
- No responsive/mobile layout — single workstation, single screen size, don't spend time on it.
- No multi-task/multi-tab management — one active task at a time is enough for the demo.
- No settings/configuration UI — model registry and thresholds are edited in the JSON config file, not through a UI.

9. Pre-build checklist

- [] Confirm the exact SSE event shape the backend will emit (field names, event types) before writing the parser — agree this with Backend Dev A in writing, don't infer it.
- [] Build the mock API stub first — this is what unblocks parallel work.
- [] Decide the color-coding convention (§7) before building any status-dependent component, so it's consistent from the first component onward.
- [] Test the "loading model..." state early — this is an easy thing to forget until the first live hot-swap demo looks broken.